

SELECTION OF QUESTIONS FROM PAST FINAL EXAM(S)

Clearly write the question number in your answer booklet and write the answer. For example the number of the following questions is 2 (a) 2 (b), 2 (c) and 2 (d).

Write short answers to each of the following questions in the answer booklet.

Q. 2: (2x5 = 10 points) State the output of the following programs. (If there is any error, clearly state the error).

```
(a) #include <iostream>
using std::cout;
class A {
    public:
    void f() const          { cout << "ff in class AA\n"; }
    virtual void g() const { cout << "gg in class AA\n"; }
    void v() const          { cout << "vv in class AA\n"; }
};

class B : public A {
    public:
    void f() const          { cout << "ff in class BB\n"; }
    virtual void g() const { cout << "gg in class BB\n"; }
    virtual void v() const { cout << "vv in class BB\n"; }
};

int main() {
    B b;
    A &r = b;
    r.f();
    r.g();
    r.v();
    return 0;
}
```

```
(b) #include <iostream>
using namespace std;
class Mystery {
    public:
    Mystery()                { cout << "Default\n"; }
    Mystery(Mystery & rhs)   { cout << "Copy\n"; }
    void operator=(Mystery & rhs) { cout << "Assignment\n"; }
    ~Mystery()               { cout << "Destructor\n"; }
};

int main() {
    Mystery m1, m4;
    Mystery m2 = m1;
    Mystery m3(m2);
    m4 = m1;
    return 0;
}
```

Q. 3: (8 points) You are given a partial implementation for inserting a **Node** in a singly linked list at the end of the list. Members of class **Node** are public.

```
class Node
{
public:
    int m_data;
    Node* m_next;
    Node(int data) : m_data{ data }, m_next{ nullptr }
    {}
};
```

Within the linked list class, the function **insertAtEnd** is partially implemented. You need to complete it so that the function works correctly. Note that you **do not have**, and **cannot introduce** a tail attribute in the linked list class.

```
class LinkedList
{
private:
    Node* m_head;
public:
    void insertAtEnd(int data)
    {
        Node* node{ new Node{ data } };
        // continue the implementation from here.
    }
};
```

Q. 4: (4 + 4 = 8 points) Consider the following class Foo that tries to keep track of the total number of objects of a class present at a given time.

<pre>// file foo.h #ifndef _FOO_H_ #define _FOO_H_ class Foo { public: Foo (); ~Foo (); static int getNumInstances() { return numInstances; } private: static int numInstances; }; #endif</pre>	<pre>// file foo.cc #include "foo.h" int Foo::numInstances = 0; Foo::Foo () { numInstances++; } Foo::~~Foo () { numInstances--; }</pre>
---	---

- (a) What is the logical flaw in the above code. In other words, what is missing which might end up in giving incorrect instance counts?
- (b) How the flaw can be fixed? Provide code that should go in foo.h and/or foo.cc file.

Q. 5: (4 + 8 = 12 points) The following class Triangle has 3 sides a, b and c. The following functions are intended as part of the class:

- Constructor which initializes a, b and c using parameters side1, side2 and side3.
- Overloaded operator == that compares two triangles by comparing their perimeters (sum of the sides) and returns true if the perimeters are equal, false otherwise.
- Overloaded operator [] that returns the side corresponding to the index, (i.e. Triangle t1; t1[0] returns a, t1[1] returns b, t1[2] returns c)

(a) (4 points) State an implementation of the == operator that works with the usage shown in main.

(b) (8 points) State an implementation of the [] operator that works with the usage shown in main.

```
#include <iostream>
```

```
using std::cout;
```

```
class Triangle
```

```
{
```

```
private:
```

```
    int a, b, c;
```

```
public:
```

```
    Triangle(int side1, int side2, int side3);
```

```
    bool operator==(const Triangle& t) const;
```

```
    int& operator[](int dim);
```

```
    void show() const;
```

```
};
```

```
Triangle::Triangle(int dim1, int dim2, int dim3) {
```

```
    a = dim1;        b = dim2;        c = dim3;
```

```
}
```

```
/* return type */ operator==(/* parameter list) { //implementation. . . }
```

```
/* return type */ operator[](/* parameter list) { //implementation. . . }
```

```
void Triangle::show() const {
```

```
    cout << a << ' ' << b << ' ' << c << '\n';
```

```
}
```

```
int main()
```

```
{
```

```
    Triangle t1(1, 2, 3), t2(4, 1, 1);
```

```
    if(t1 == t2) // if(t1.operator==(t2))
```

```
        cout << "Same perimeters\n";
```

```
    else
```

```
        cout << "Different perimeters\n";
```

```
    t1[0] = 4;    //Assigning t1.a = 4
```

```
    t1[1] = 5;    //Assigning t1.b = 5
```

```
    t1[2] = 6;    //Assigning t1.c = 6
```

```
    t1.show();
```

```
    return 0;
```

```
}
```

Q. 6: (6 points) Data Hiding is a fundamental concept of Object-Oriented Programming, where member attributes are declared as private and accessed through getters and setters. If getters and setters allow both reading and writing an attribute's value, why is it preferred to keep the attribute private instead of declaring it as public?

Q. 7: (6 points) Consider a class whose name is B. We want to write another class D for which there are several options: make D a friend class of B, derive D from B, do both, etc. Fill in the following table by writing Y (for yes) or N (for no) in each square (Construct and fill the table in your answer booklet):

	Methods in D can access public members of B	Methods in D can access protected members of B	Methods in D can access private members of B
<pre>class B {...}; class D : public B {...};</pre>			

[Section 3] Programming Questions [30 points]

For each of the following questions, you have to write functions/classes in your answer booklet. Clearly write the question number you are attempting. It is preferable to begin each question on a new page.

Q. 8: (15 points) Design a Stack template class that maintains a static counter for the total number of items currently stored in all instances of Stack. This counter should be updated as elements are added to or removed from any stack instance.

You must also Implement the following within your template class:

- A static function `getTotalNumberOfInstances` that returns the total number of instances currently used across all stack instances.
- A constructor that initializes the stack with a given size.
- A push function that adds an element to the stack.
- A pop function that removes the top element.
- An `isEmpty` function that checks if the stack is empty or not.
- A constant peek function to view the top element without modifying the stack.
- Any necessary destructors to manage memory.

Your implementation should maximize and maintain constant correctness.